

Sisteme de operare

Laborator 9

Semestrul I 2025-2026

Laborator 9

- comunicare inter-procese (IPC)
- System V IPC
 - semafoare
 - memorie partajata

Semafoare System V

- complica paradigma generala
 - lucreaza cu set-uri de semafoare
 - crearea unui semafor (set, de fapt) e independenta de initializarea sa
 - consecinta: nu putem crea si initializa atomic un semafor
 - ca orice structura de date System V IPC continua sa existe in kernel si dupa ce nu mai exista procese care sa o utilizeze
 - daca un program termina fara sa elibereze semafoarele pe care le-a alocat, avem nevoie de o operatie de *undo*
- un semafor este reprezentat de o structura cu urmatoarele campuri
 - *semval* – valoarea semaforului, ≥ 0
 - *sempid* – PID-ul procesului care a efectuat ultima operatie asupra semaforului
 - *semncnt* – nr proceselor care asteapta ca *semval* $>$ valoarea curenta
 - *semzcnt* – nr proceselor care asteapta ca *semval* $= 0$

Creare & intializare semafoare

- creare

```
int semget(key_t key, int nsems, int flag);
```

- *key* se obtine cu `IPC_PRIVATE` sau *ftok*
- *nsems* = nr semafoarelor din set (*nsems* = 0, acces la un set de semafoare existent)
- intoarce un *semid*

- initializare

```
int semctl(int semid, int semnum, int cmd, union semun arg);
```

```
union semun {
```

```
    int val;                // setare valoare semafor
```

```
    struct semid_ds *buf;    // acces/setare structura de date semafor
```

```
    ushort *array;          // GETALL si SETALL pe setul de semafoare
```

```
}
```

Comenzi semctl

- IPC_STAT – citire structura de date semafor
- IPC_SET – setare permisiuni (UID/GID, mode)
- IPC_RMID – sterge semaforul din sistem imediat, alte procese care incearca operatii asupra semaforului primesc EIDRM
- GETVAL – intoarce valoarea *semval*
- SETVAL – seteaza valoarea *semval* la *arg.val*
- GETALL – citește toate valorile *semval* din setul de semafoare in *arg.array*
- SETALL – seteaza toate valorile *semval* din setul se semafoare la valorile din *arg.array*
- `semctl`

Operatii cu semafoare

- folosesc o structura de date de tipul

```
struct sembuf {  
  
    ushort sem_num;           // nr semafor in set: 0, 1 , ...  
  
    short sem_op;            // operatia > 0, 0, < 0  
  
    short sem_flg;          // IPC_NOWAIT, SEM_UNDO  
  
    }
```

```
int semop(int semid, struct sembuf semoparray[], size_t nops);
```

- executa **atomic** cele *nops* operatii din *semoparray*

Semnificatia `sem_op`

- valoare pozitiva (eliberare resurse)
 - valoarea lui `sem_op` este adaugata la `semval`
 - pt `SEM_UNDO`, `sem_op` se scade din valoarea de ajustare a semaforului pt acest proces
- valoare negativa (alocare resurse)
 - $semval \geq abs(sem_op) \Rightarrow$ se scade $abs(sem_op)$ din `semval`
 - conditia garanteaza ca `semval` ramane ≥ 0 !
 - la `SEM_UNDO`, $abs(sem_op)$ se adauga la valoarea de ajustare a semaforului pt acest proces
 - $semval < abs(sem_op) \Rightarrow$ resursele nu sunt disponibile
 - pt `IPC_NOWAIT`, `semop` se intoarce cu eroare setata pe `EAGAIN`
 - fara `IPC_NOWAIT`, `semncnt` se incrementeaza si procesul apelant se suspenda pana cand:
 - $semval \geq abs(sem_op)$, `semncnt` se decrementeaza si se scade $abs(sem_op)$ din `semval`; la `SEM_UNDO`, $abs(sem_op)$ se adauga la valoarea de ajustare a semaforului pt acest proces
 - semaforul este sters din sistem $\Rightarrow$ `semop` intoarce `ERMID`
 - procesul primeste un semnal si handlerul de semnal se intoarce $\Rightarrow$ `semncnt` se decrementeaza si `semop` se intoarce cu `EINTR`

Semnificatia sem_op (cont.)

- valoare nula
 - daca *semval* == 0, *semop* se intoarce imediat
 - daca *semval* != 0
 - pt IPC_NOWAIT, *semop* intoarce EAGAIN
 - fara IPC_NOWAIT, *semzcnt* este incrementata si procesul apelant se suspenda pana cand
 - i) *semval* devine 0; *semzcnt* se decrementeaza
 - ii) semaforul este sters din sistem => *semop* intoarce ERMID
 - iii) procesul primeste un semnal si handlerul de semnal se intoarce => *semzcnt* se decrementeaza si *semop* se intoarce cu EINTR

Ajustarea semaforului la *exit*

- cand specificam SEM_UNDO si alocam resurse ($sem_op < 0$) kernelul memoreaza nr resurselor alocate, i.e. $abs(sem_op)$
- la terminarea procesului, kernelul verifica daca exista ajustari de facut pt semafoarele procesului si le aplica
- daca setam valoarea semaforului cu *semctl* si SETVAL/SETALL valoarea de ajustare e setata pe 0 pt toate procesele

Exemplu utilizare semafoare Sys V

- initializare semafor

```
union semun arg;
```

```
int semid;
```

```
semid = semget(IPC_PRIVATE, 1, IPC_CREAT | 0644);
```

```
if(semid < 0)
```

```
    perror("semget"), exit(1);
```

```
arg.val = 1;
```

```
if(semctl(semid, 0, SETVAL, arg) < 0)
```

```
    perror("semctl SETVAL"), exit(1);
```

Exemplu down

```
struct sembuf down;
```

```
down.sem_num = 0;
```

```
down.sem_op = -1;
```

```
down.sem_flg = 0;
```

```
if(semop(semid, &down, 1) < 0)
```

```
    exit(1);
```

Exemplu up

```
struct sembuf up;
```

```
up.sem_num = 0;
```

```
up.sem_op = 1;
```

```
up.sem_flg = 0;
```

```
if(semop(semid, &up, 1) < 0)
```

```
    exit(1);
```

Memorie partajata

- permite mai multor procese sa partajeze o regiune de memorie
- cea mai rapida forma de IPC pentru ca nu e nevoie de copierea datelor
- dezavantaj: procesele care partajeaza regiunea de memorie trebuie sa se sincronizeze
 - accesele la regiunea de memorie partajata trebuie executate in sectiune critica
- creare

int shmget(key_t key, int size, int flag);

- *key* generat cu IPC_PRIVATE sau *ftok*
- *size* = dimensiunea regiunii de memorie partajata (*size* = 0, acces la o regiune de memorie partajata existenta)

Atasarea memoriei partajate

*int shmat(int shmid, void *addr, int flag);*

- adresa din proces la care se ataseaza regiunea de memorie depinde de valorile ultimilor doi parametri de apel
 - 1) *addr* = 0, kernelul ataseaza regiunea de memorie la prima adresa disponibila (metoda recomandata)
 - 2) *addr* != 0 si SHM_RND nu apare in flag => regiunea de memorie este atasata la adresa *addr*
 - 3) *addr* != 0 si SHM_RND setat => regiunea se ataseaza la adresa *addr* rotunjita la urmatorul multiplu al SHMLBA (*low boundary address multiple*)
- *flag* = SHM_RDONLY atasarea se face RO, altfel RW
- valoarea de retur: este adresa din proces la care a fost atasata regiunea (-1 in caz de eroare)

Detasarea memoriei partajate

*int shmdt(void *addr);*

- Obs: detasarea nu sterge ID-ul memoriei partajate si nici structura de date aferenta
 - supravietuiesc pana cand se sterge zona de memorie partajata
- *addr* este valoarea returnata de un apel *shmat* anterior

Operatii cu memorie partajata

- executate cu *shmctl*

*int shmctl(int shmid, int cmd, struct shmid_ds *buf);*

Valori *cmd*: IPC_STAT – citeste in *buf* structura de date asociata in kernel regiunii de memorie

IPC_SET – seteaza permisiunile (UID/GID si *mode* in structura de data asociata in kernel regiunii de memorie)

IPC_RMID – sterge regiunea de memorie din sistem imediat, a.i orice operatie subsecventa de atasare esueaza

Obs: totusi regiunea de memorie nu dispare din kernel pana cand ultimul proces atasat la ea nu a terminat (sau se detaseaza de ea)

SHM_LOCK/SHM_UNLOCK - *pinned down memory*, memoria nu poate fi swappata pe disc (doar *root*-ul poate executa operatia de *pin down*)

Sisteme de operare

Laborator 9

Semafoare si memorie partajata System V

1. Creati un nou subdirector *lab9/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu *PATH* pentru a include directorul *SO/laborator/lab9/bin*.

2. Scrieti un program C **mem-layout.c** care tipareste adresele (aproximative) ale sectiunilor unui program: cod (.text), date neinitializate (.bss), date initializate, stiva, heap (+ program break), memorie partajata. Pentru a realiza acest lucru, tipariti urmatoarele adrese:

- adresa functiei *main*
- adresa *argv*
- adresa unei variabile globale initializate
- adresele de inceput si sfarsit ale unui array global de octeti neinitializat (de dimensiune relativ mare, sa zicem 40000 bytes)
- adresa primei variabile locale declarate in *main* (va fi alocata pe stiva)
- adresa unui vector de 10000 octeti alocat in *main* cu *alloca*
- adresa unui vector de 10000 de octeti alocat dinamic in *main* cu *malloc* + program break-ul (folositi apelul *sbrk*)
- creati o zona de memorie partajata privata (*shmget* cu *IPC_PRIVATE*) si tipariti valoarea adresei la care se ataseaza in program (obtinuta cu *shmat*).

La final, apelati *shmctl* cu *IPC_RMID* pentru a sterge zona de memorie partajata din sistem.

Ce valoare au cele mai mari adrese tiparite de program? Calculatorul pe care lucrati cata memorie RAM are? Cum va explicati situatia?

Indicatie: adresele de variabile se tiparesc folosind *%p* in formatul *printf*.

3. Rescrieti programul C **race.c** din laboratorul 4 in care doua procese, parinte si copil, scriu concurent propriul string pe ecran, scrierea fiind facuta caracter cu caracter. Pentru a evidentia mai clar race condition-ul, procesele parinte si copil repeta operatia de tiparire pe ecran de un numar de ori, sa zicem de 10 ori, in bucla. Folositi un semafor System V pentru a impiedica producerea race condition-ului si a obtine un output neintretesut pe ecran. La final, apelati *semctl* cu *IPC_RMID* pentru a sterge semaforul din sistem.

Indicatie: fiind vorba de procese inrudite, puteti crea semaforul cu *IPC_PRIVATE*. De asemenea, e util sa scrieti functii ajutatoare pentru implementarea operatiilor *down* si *up* cu semnificatiile de la curs.

Folositi aceasta ocazie pentru a invata compilarea conditionata. Portiunile de cod pe care le adaugati la **race.c** pentru a rezolva race condition-ul se vor executa doar daca e definit flag-ul *FIXED*, ca mai jos:

```
#ifdef FIXED
    // cod semaforizat pentru protectia apelului myprint
#endif
    myprint(...)
```

În acest fel, aveți posibilitatea de a compila condiționat **race.c** și a testa ambele versiuni ale programului, cea care evidențiază race condition, și cea care rezolvă problema, ca mai jos:

```
$ gcc -o race race.c
$ race                                # executia cu race condition
$ gcc -o race race.c -DFIXED
$ race                                # executia corecta, care rezolva
                                         # problema
```

4. Scrieți un program C **sync.c** în care un proces părinte creează două procese copil și apoi așteaptă terminarea executiei lor. Primul proces copil doarme pentru o perioadă de 5 secunde după care afișează pe ecran mesajul *“first child finished\n”* și termină executia. Al doilea proces copil doarme pentru o perioadă de 1 secundă după care afișează pe ecran mesajul *“second child finished\n”* și termină executia. Folosiți semafoare System V pentru a vă asigura că indiferent de planificarea proceselor în sistem, primul proces copil va tipări întotdeauna mesajul sau înaintea celui de-al doilea.

5. Scrieți un program C **shm.c** care creează o zonă de memorie partajată între un proces părinte și un copil. Părintele creează zonă de memorie partajată (puteți folosi `IPC_PRIVATE`) și o atașează la proces înainte de apelul *fork*. Procesul copil execută un loop cu 10 iterații în care scrie succesiv, la fiecare iterație, următorul mesaj în zonă de memorie partajată:

"this is the child printing " + numărul iterației

La sfârșitul fiecărei iterații, procesul copil așteaptă ca procesul părinte să citească mesajul pe care tocmai l-a scris, și abia apoi continuă cu următoarea iterație.

Pentru a compune mesajele procesului copil puteți folosi funcția de bibliotecă *sprintf*. Mesajele din fiecare iterație se adaugă (*append*) la sfârșitul mesajului scris în iterația anterioară (cu excepția primei iterații).

Procesul părinte execută și el un loop cu 10 iterații în care, la fiecare iterație, așteaptă ca procesul copil să scrie un mesaj în buffer, citește mesajul și îl afișează pe ecran.

Dezvoltați o soluție folosind semafoare System V care să permită coordonarea (sincronizarea) accesului celor două procese la zonă de memorie partajată a.i. executia programului să fie cea anticipată, și anume după fiecare mesaj scris de copil în memoria partajată părintele îl citește și îl scrie pe ecran. (*Obs: ambele procese execută un loop cu 10 iterații dar nu puteți face nici o presupunere asupra vitezei relative de executie a celor două procese*).

La final, apăsați *shmctl* și *semctl* cu `IPC_RMID` pentru a șterge memoria partajată și semafoarele din sistem.

6. Modificați programele **ping.c** și **pong.c** din laboratorul 3. Programele comunică printr-un fișier FIFO pe care îl deschid atât pentru scriere cât și pentru citire. Programul **ping.c** execută 5 iterații, în fiecare dintre ele scriind mesajul *“ping\n”* în FIFO și apoi citește din FIFO răspunsul programului **pong.c** pe care îl afișează pe ecran. Programul **pong.c** execută și el 5 iterații, în fiecare iterație citind un mesaj din FIFO pe care îl afișează pe ecran iar apoi răspunde cu mesajul *“pong\n”* pe care îl scrie în FIFO.

Ce se întâmplă? Cum puteți rezolva problema?

7. Folositi semafoare si memorie partajata System V pentru a implementa problema producator-consumator. Cele doua procese trebuie sa fie neinrudite (i.e., nu sunt create cu *fork*). Buffer-ul circular va fi implementat folosind o zona de memorie partajata.